Field Hours

CSCE 637 Complexity Theory

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Polynomial time, measured in n

You place a language in P when a deterministic Turing machine decides it in at most c times n to the k steps, for fixed c and k. Even parity is one pass. On 101011, length 6, you flip state once per 1. There are 4 ones, an even count, so you accept by step 6. The exponent is 1. A bound that is not a fixed power of n is not a membership proof for P.

You measure n in symbols of the input, not in the integer a binary string denotes. Trying all 2^m strings of length m is exponential in the length. For m = 10 you try 1024 strings, while m squared is only 100. When you claim a problem is in P, you name the machine and the exponent. A machine that always halts can still sit outside P. Halting is not the same promise as a polynomial budget.

Even parity in one pass

1. Read the next bit
2. Flip state on a 1
3. Stop at the blank
4. Accept when the count is even

Parity of 101011

1. The string 101011 has length 6, and the 1 bits are in positions 1, 3, 5, and 6.
2. You flip four times. Four is even, so you accept.
3. The scan finishes by step 6. That is n to the first power.

Problem 1. Input 111000 has length 6. Do you accept, and how many flips do you make?

Answer. There are 3 ones, an odd count, so you reject. You flip 3 times and finish by step 6. Rejection is still inside P.

Problem 2. A procedure tries every binary string of length m = 10. How many strings is that, and why is the procedure outside a bound of m squared?

Answer. 2^10 = 1024 strings. m squared is 100. The loop is already past a quadratic budget at this length, and the gap grows.

Lecture 2 · 90 minutes

A certificate you can check

You place a language in NP when every yes input has a certificate that a deterministic checker accepts in polynomial time. For a boolean formula the certificate is an assignment. The formula (x or y) and (not x) has certificate x = 0, y = 1. You check 2 clauses. The first is true because y is 1. The second is true because x is 0. You do not try the other 3 assignments of 2 bits.

You do not get a short certificate for a no input. That asymmetry is the content of NP. Even parity on length 6 is in P, hence in NP: the checker ignores any certificate and runs the 6-step scan. Membership in NP is the easy direction for a problem already in P. The interesting direction is a short check for a yes answer when you do not know a short way to find it.

Four rows versus one witness

All 4 assignments

00, 10, and 11 fail

Only 01 succeeds

One certificate

Bits x = 0, y = 1

Check the 2 clauses

Checking x = 0, y = 1

1. Clause (x or y) becomes (0 or 1), which is true.

2. Clause (not x) becomes not 0, which is true.

3. Both clauses pass. You evaluated 1 assignment, not all 4.

Problem 1. Is x = 1, y = 1 a certificate for (x or y) and (not x)?

Answer. No. The first clause is true, but (not x) is false. The checker rejects this pair. The passing certificate is x = 0, y = 1.

Problem 2. Why is even parity on an input of length 6 in NP?

Answer. A language in P is in NP. The checker runs the 6-step scan and may ignore the certificate. An even count is verified in polynomial time.

Lecture 3 · 90 minutes

Reductions and a 4-cycle

You reduce A to B by a polynomial-time function f such that x is in A exactly when f(x) is in B. If B is in P, you decide A by computing f and running B's decider. Hardness reductions run from a problem you already know is hard, toward the problem you care about. Reducing an easy problem to a hard one proves nothing about the hard one. You can encode parity into a hard problem on 4 vertices.

You can check one reduction on 4 vertices. Let the edges be 1-2, 2-3, 3-4, and 4-1, a 4-cycle. A set S is independent exactly when V minus S is a vertex cover. Take S = {1, 3}. No edge has both ends in S, so S is independent and has size 2. The complement {2, 4} hits every edge and has size 4 - 2 = 2. One construction

answers both questions. You did not search the graph twice.

One subset, two names

Independent set

S = {1, 3}

Size 2, no inside edge

Vertex cover

Complement is {2, 4}

Size 2, every edge hit

Cover from the complement

1. The graph is a 4-cycle: edges 1-2, 2-3, 3-4, and 4-1.

2. S = {1, 3} contains no edge. It is an independent set of size 2.

3. The complement {2, 4} meets all 4 edges. Its size is 4 - 2 = 2.

Problem 1. Is S = {1, 2} independent in this 4-cycle, and is its complement a vertex cover?

Answer. No. {1, 2} contains edge 1-2. The complement is {3, 4}, which contains edge 3-4, so it is not a cover. The two failures match.

Problem 2. You reduce parity to vertex cover in polynomial time. Have you shown parity is NP-hard?

Answer. No. A reduction from A to B says B is at least as hard as A. Parity is in P. Hardness of vertex cover needs a reduction from a known hard problem into vertex cover.

Lecture 4 · 90 minutes

Space, squared by Savitch

You place a language in PSPACE when a Turing machine decides it using polynomially many tape cells, reusing cells as time goes on. Time may be exponential while the cell count stays modest. Savitch's theorem says a nondeterministic machine that uses s cells can be simulated deterministically in on the order of s squared cells. You square the space, so 5 cells become 25. You do not keep the running time.

You get the square from recursive reachability. To ask whether configuration A reaches B within t steps, you name a middle configuration M and ask two questions of length t/2. The depth is logarithmic in the number of configurations, and that number is exponential in s, so the depth is proportional to s. Each level stores one configuration of s cells. With s = 5 the product is on the order of 25 cells.

Split reachability at the middle

Ask if A reaches B

Name a middle M

Two questions of half length

Store s cells per level

Five cells become twenty-five

1. The nondeterministic machine uses s = 5 cells.

2. The number of configurations is exponential in 5, so the recursion depth is proportional to 5.

3. Space is on the order of 5 times 5 = 25 cells. Time may grow much faster than that. You squared space, not time.

Problem 1. A nondeterministic machine uses s = 4 cells. On the order of how many cells does the deterministic simulation use?

Answer. 4 squared = 16 cells. The theorem squares the space bound. It does not say the simulation runs in 16 steps.

Problem 2. Why store the middle configuration instead of the whole path?

Answer. The path can be exponential in s. Storing it would break the space bound. One middle configuration, with the space reused on the two half questions, costs s cells per level.

Lecture 5 · 90 minutes

One-sided error, repeated

You place a language in RP when a randomized polynomial-time machine has one-sided error. On a no input it always says no. On a yes input it says yes with probability at least 1/2. Independent repetition cuts the miss probability. One yes trial fails with probability at most 1/2, so 2 failures have probability at most 1/4, and 3 failures have probability at most 1/8. A no input still never produces a yes.

You treat BPP as two-sided error at most 1/3 on every input, cut down by a majority of repeated trials. RP is the one-sided case. Neither class is known to leave P, and you are not asked to settle that. After 3 RP trials on a yes input, the chance that every trial missed is at most 1/8, so the chance you saw a yes is at least 7/8.

Repeat an RP trial

Run one trial -> Accept on the first yes -> Otherwise repeat -> Stop by trial 3

Three misses at one eighth

1. On a yes input, one trial fails with probability at most 1/2.

2. Two independent failures have probability at most 1/4. Three have probability at most 1/8.

3. The chance of at least one yes by trial 3 is at least 1 - 1/8 = 7/8. A no input still yields no yes answers.

Problem 1. An RP yes input is tested twice, and one trial fails with probability at most 1/2. What is the probability both fail?

Answer. At most 1/4. Independent trials multiply. You do not add the two probabilities.

Problem 2. Why can repetition not turn an RP no input into a yes?

Answer. On a no input every trial says no. Repeating those no answers still yields no. The fraction 1/8 applies only to missing a yes input.

Lecture 6 · 90 minutes

AND states and OR states

You read an alternating machine as AND states and OR states. An OR state accepts if some move accepts. An AND state accepts if every move accepts. Alternating polynomial time decides exactly PSPACE, the class whose deterministic space Savitch squared. A quantified boolean formula is the same idea in logic. Exists is an OR over a bit. Forall is an AND over a bit. You evaluate the 2 quantifiers from the outside.

You evaluate exists x forall y of (x or not y) on single bits. Pick x = 1. Then both y = 0 and y = 1 make the clause true, so the AND accepts on 2 values and the OR accepts. Pick x = 0 and y = 1 fails, so that branch dies. One good x is enough. You checked 2 values of y.

Two choices of the bit x

Pick x = 1

y = 0 succeeds

y = 1 succeeds

Pick x = 0

y = 0 succeeds

y = 1 fails

A sentence that is true

1. Choose x = 1. For y = 0 the clause (1 or not 0) is true.

2. For y = 1 the clause (1 or not 1) is true. Both AND branches accept.

3. Exists therefore succeeds. You do not need the failing branch x = 0.

Problem 1. Evaluate exists x forall y of (x and y) on one-bit variables.

Answer. False. For x = 1, y = 0 makes (1 and 0) false, so forall fails. For x = 0 both values of y fail. Exists finds no good x.

Problem 2. When you accept exists x forall y of (x or not y) by picking x = 1, how many values of y do you still check?

Answer. Both of them, 2 values. Exists lets you ignore x = 0, but the inner forall still requires y = 0 and y = 1.

Lecture 7 · 90 minutes

Oracles the diagonal does not cross

You say a proof relativizes when it still works after every machine in it receives the same oracle, a box that

answers one membership question in 1 step. There is an oracle that makes P and NP equal, and a different oracle that separates them. A proof that P equals NP, or that it does not, needs an ingredient that fails for at least one of those oracles. A technique that survives every oracle cannot finish the question alone.

You can still diagonalize a time bound. That argument relativizes. Build a language decided in about n squared steps that no n-step machine decides. For n = 4, simulate the fast machine for 16 steps and flip its bit. It is wrong on that input. The flip does not separate P from NP: an oracle can add questions the simulation never counts. You keep the hierarchy. You do not promote it.

Flip one fast machine

1. Take an input of length 4

2. Simulate for 16 steps

3. Output the opposite bit

4. That fast machine is wrong

Budget 4 against budget 16

1. An n-step machine on n = 4 has a budget of 4.

2. The diagonal language may use about n squared steps, here 16, which is enough to see the fast answer.

3. You write the opposite bit. This fast machine and the diagonal language disagree on the input.

Problem 1. For n = 5, how many steps does the n-squared simulation get, and why is that enough to flip an n-step machine?

Answer. 5 squared = 25 steps. The fast machine stops by step 5, so 25 steps can read its answer and write the opposite bit.

Problem 2. Why does the same flip not prove that P differs from NP?

Answer. Some oracles make P and NP equal, and some separate them. A proof that treats every oracle the same way cannot give both outcomes. The flip separates time budgets. It does not, alone, separate P from NP.

Lecture 8 · 90 minutes

What you can now place

You can now put a language in P with a power of n, put it in NP with a certificate, and carry hardness along a polynomial reduction. You can square a space bound, cut an RP miss probability from 1/2 to 1/8 in 3 trials, evaluate two quantifiers, and say why a relativizing flip does not settle P against NP. On the 4-cycle, the independent set {1, 3} and the cover {2, 4} are one fact said twice.

You match the claim to the witness. Certificate x = 0, y = 1 shows a formula is in NP: the checker reads 2 clauses. It does not show NP-hardness, which needs a reduction from a known hard problem. For n = 4 the hierarchy simulation used 16 steps. Savitch on s = 4 uses about 16 cells. Those two copies of 16 are not the same resource.

Two different sixteens

Time hierarchy

n = 4 gives 16 steps

Flip the fast answer

Savitch space

s = 4 gives 16 cells

Square the space bound

A certificate is not completeness

1. Certificate x = 0, y = 1 makes both clauses of (x or y) and (not x) true. That places satisfiability in NP.

2. Separately, n = 4 yields 16 simulation steps, and s = 4 yields about 16 cells.

3. The shared numeral 16 does not identify steps with cells, and the 2-bit certificate does not prove NP-completeness.

Problem 1. A 2-bit certificate passes a 2-clause check. Have you proved the language is NP-complete?

Answer. No. You have membership in NP. Completeness also needs a reduction from a known NP-complete problem, or from every language in NP. The certificate is only the membership direction.

Problem 2. Combine the hierarchy with Savitch. For n = 4 and s = 4 both calculations produce 16. What does each 16 count, and does the match identify time with space?

Answer. The hierarchy gets 16 steps, from 4 squared. Savitch gets about 16 cells, from 4 squared. Steps are not cells. Equal numerals do not make the resources the same.